Multi-agent service platform working :

Structure of the project :

```
multi-agent-service-platform/
│
├── app/                    # Backend API & Agent Logic
│   ├── main.py             # FastAPI entry point
│   ├── config.py           # Environment configurations & settings
│   ├── agents/             # LangChain & LangGraph agents
│   │   ├── supervisor_agent.py
│   │   ├── email_agent.py
│   │   ├── reminder_agent.py
│   │   ├── composer_agent.py
│   │   ├── assistant_agent.py
│   │   └── document_agent.py
│   ├── models/             # ML Models (Spam Classifier)
│   │   └── train_classifier.py
│   ├── routes/             # API Endpoints
│   │   ├── agent_routes.py
│   │   ├── email_routes.py
│   │   └── media_routes.py
│   └── utils/              # Helper modules and integrations
│       ├── calendar_helper.py
│       ├── gmail_helper.py
│       ├── logger.py
│       ├── notifier.py
│       ├── vector_db.py
│       └── voice_helper.py
│
├── frontend/               # Streamlit UI
│   └── app.py
```

```
│
├── .env                  # API Keys & Secrets
├── credentials.json      # Google Cloud OAuth Credentials
└── requirements.txt      # Python dependencies
```

Note : watch the code alongside to understand better

Multi-Agent Service Platform

Architecture, Technology, and Workflow Documentation

Overview:

The Multi-Agent Service Platform is an AI-powered, voice-enabled office automation system. By leveraging a multi-agent framework, the system autonomously orchestrates and executes daily administrative tasks, including inbox management, calendar scheduling, and proprietary document analysis, all triggered via natural language voice commands.

System Architecture

The platform operates on a decoupled client-server architecture to ensure high performance during intensive AI processing tasks while maintaining a responsive user interface.

- **Frontend (The Face):** A lightweight Streamlit web application that captures user interactions, records audio commands, processes document uploads, and routes them as API requests to the backend server.
- **Backend (The Brain):** A robust FastAPI server housing the LangGraph Supervisor. This node acts as the central intelligence hub, managing all API endpoints, orchestrating specialized agent workflows, and maintaining system state.
- **Orchestration Layer:** A LangGraph state machine evaluates user intent and dynamically routes tasks to the correct specialized agent, ensuring that complex, multi-step queries are handled systematically.

Technical Stack

- **Core Frameworks:** FastAPI (Backend API), Streamlit (Frontend UI)
- **AI & Orchestration:** LangChain, LangGraph, OpenAI (`gpt-4o-mini`), Whisper (Audio Transcription)
- **Machine Learning:** Python, `scikit-learn`, `pandas`, `joblib` (Custom Spam Classification Model)
- **Database & Storage:** ChromaDB (Local Vector Store for RAG architecture)
- **External APIs:** Google Workspace (Gmail API, Calendar API v3), Twilio API (WhatsApp Notifications)

Agent Workflows

The platform relies on a network of specialized agents, each strictly scoped to perform specific operational duties.

1. The Orchestrator (Supervisor Agent)

Serves as the central routing engine. It receives transcribed voice commands, classifies the primary intent into strict categories (`RESEARCH`, `SUMMARIZE_EMAILS`, `SCHEDULE_EVENT`, `COMPOSE`), and dictates the execution path across the entire platform.

2. Email & Notification Agent

Automates inbox management through a multi-stage data pipeline:

- **Retrieval:** Securely fetches unread messages via the Gmail API.
- **Classification:** Evaluates emails against a custom local machine learning model to categorize them as casual, important, or spam, immediately dropping spam records.
- **Summarization:** Leverages LangChain to generate concise summaries of legitimate emails, prioritized by importance.
- **Notification:** Dispatches the final prioritized digest directly to the user's mobile device via Twilio WhatsApp integration.

3. Reminder & Calendar Agent

Transforms natural language requests into structured chronological events:

- **Temporal Parsing:** Dynamically calculates relative dates (e.g., "tomorrow at 4 PM") against the current system time (Asia/Kolkata) to generate precise ISO 8601 timestamps.
- **Execution:** Constructs the required JSON payload, books the meeting via the Google Calendar API, and triggers the Notification Agent to send a WhatsApp confirmation link to the user.

4. Document (RAG) Agent

Manages the local knowledge base for rapid proprietary information retrieval:

- **Ingestion:** Extracts raw text from uploaded PDFs, chunks the data into semantic blocks, and stores it as queryable vector embeddings in ChromaDB.
- **Retrieval:** Answers user queries by referencing local vectors. If the required context is unavailable locally, the agent initiates a fallback to integrated web search tools.

5. Composer Agent

Synthesizes context to draft professional correspondence. It combines the user's explicit voice instructions with factual data retrieved from the Document Agent to output accurately formatted, context-aware email drafts.